

4. Machine Learning Model

This chapter describes the machine learning component of the Dublin Bikes project. The model predicts the number of available bikes at any station for each weather forecast entry cached in the database (up to 48 hours ahead). Training code and exported artefacts (`bike_availability_model.pkl`, `model_features.pkl`) are stored under `flask-app/machine_learning/`.

4.1 Problem Definition

The goal is to help users decide when and where to pick up a bike by forecasting future availability. The problem is framed as a supervised regression task:

- Target variable: `num_bikes_available` — the number of bikes that can be rented from a station at a specific point in time.
- Inputs: a vector of temporal, spatial, and meteorological features that are known in advance or can be observed at prediction time.

Predictions are generated in bulk for every forecast entry stored in the `weather_forecast` table whose `forecast_time` is at or after the current hour. Results are exposed to the frontend via the GET `/api/stations/<number>/prediction` endpoint, where `<number>` is the JCDecaux station number used as the primary key in the station table.

4.2 Dataset and Feature Engineering

4.2.1 Raw Data

The training set is `final_merged_data.csv` (located in `flask-app/machine_learning/`), containing 298,946 rows and 78 columns. It was built by joining historical station-availability snapshots with historical weather readings provided by Met Éireann. Each row represents the state of one specific station at one specific hour, augmented with the weather conditions recorded during that hour. An initial exploratory analysis confirmed that the dataset contains no missing values, so no imputation step was required.

4.2.2 Feature Selection

The following column groups were removed:

Removed Columns	Reason
<code>*_quality_indicator</code>	Metadata flags with no meteorological signal
<code>*_std_deviation</code>	Redundant once min/max values are retained
Soil/grass/earth temperature	Low correlation with urban bike usage
Text columns (name, address, etc.)	Location encoded by <code>station_id</code> , <code>lat</code> , <code>lon</code>
Near-constant booleans (<code>is_installed</code> , etc.)	Almost 100% True; no discriminative power

4.2.3 Engineered Features

To capture periodic and meteorological patterns more explicitly, five derived features were created from the reduced column set:

- `day_of_week` — integer 0 (Monday) to 6 (Sunday), extracted from the date column.
- `is_weekend` — binary indicator (1 if Saturday or Sunday, 0 otherwise).

- `avg_temperature` — arithmetic mean of `max_air_temperature_celsius` and `min_air_temperature_celsius`.
- `avg_humidity` — arithmetic mean of `max_relative_humidity_percent` and `min_relative_humidity_percent`.
- `avg_pressure` — arithmetic mean of `max_barometric_pressure_hpa` and `min_barometric_pressure_hpa`.

The averaged values were preferred over raw min/max pairs to reduce collinearity while preserving the same physical signal. A correlation check also revealed that month and year had zero variance across the dataset (which covers December 2024 only); both were dropped as they cannot generalise to other months.

4.2.4 Final Feature Set (11 features)

The model learns from the following 11 input features, in the exact order required by the serialised pipeline:

- `station_id` — the JCDecaux station number (primary key in the station table).
- `capacity` — total number of docking stands at the station.
- `lat / lon` — station latitude and longitude.
- `hour` — hour of day (0–23); `day` — day of month (1–31).
- `day_of_week` — weekday index (0–6, Monday = 0); `is_weekend` — weekend flag (0/1).
- `avg_temperature` (°C), `avg_humidity` (%), `avg_pressure` (hPa).

Note: at inference time, the `avg_*` features are populated from hourly OpenWeatherMap point forecasts rather than daily min/max aggregates. The column names are kept identical so that the ordering matches `model_features.pkl`, but the underlying distributions differ slightly from those seen during training.

4.3 Training and Evaluation

The cleaned dataset was split using scikit-learn's `train_test_split` with a 70/30 ratio and `random_state=42` for reproducibility, yielding 209,262 training rows and 89,684 test rows. Four regression algorithms were trained and evaluated under identical conditions:

- Linear Regression — a simple baseline to establish a lower bound on performance.
- Decision Tree Regressor — captures non-linear interactions without requiring feature scaling.
- Random Forest Regressor — an ensemble of decision trees, configured with `n_estimators=100`, `max_depth=15`, and `min_samples_leaf=10` for modest regularisation.
- Gradient Boosting Regressor — builds trees sequentially to correct residuals from previous iterations.

Performance was measured with three standard metrics: Mean Absolute Error (MAE), Root Mean Squared Error (RMSE), and the Coefficient of Determination (R^2).

Model	MAE	RMSE	R^2
Linear Regression	7.829	9.366	0.075
Decision Tree	0.970	2.376	0.941
Random Forest (selected)	2.325	3.393	0.879
Gradient Boosting	6.120	7.537	0.401

Linear Regression performed poorly ($R^2 \approx 0.08$), confirming that bike availability is not linearly separable. The Decision Tree achieved the lowest MAE (0.970) and highest R^2 (0.941), but its perfect fit on training data raised concerns about overfitting. The Random Forest Regressor was therefore selected as the final model: it sacrifices a small amount of accuracy compared to the Decision Tree while offering meaningfully better generalisation and resistance to noise.

4.4 Production Integration

The trained Random Forest and the ordered feature list were serialised with pickle as `bike_availability_model.pkl` and `model_features.pkl` respectively. At inference time, the Flask service constructs a pandas DataFrame with the 11 required features, calls `model.predict()`, and clamps each output to the range `[0, station.bike_stands]` to prevent physically impossible values.

The most important features, in descending order of contribution, were: `lat/lon` (geographic position), `day` and `hour` (calendar specificity), `station_id`, `avg_pressure`, `capacity`, `day_of_week`, `avg_temperature`, `avg_humidity`, and `is_weekend`.

4.5 Reflection and Future Work

Strengths

- The pipeline is straightforward and fast to retrain — no missing-value imputation is needed and the feature set is compact.
- Predictions are directly actionable: the output is a bike count, not an abstract score, making it easy to display to end users.
- The REST endpoint decouples the model from the frontend, allowing both to be updated independently.

Limitations

- Temporal scope: training data covers only December 2024. Seasonal variation (summer vs. winter commuting patterns) is not captured, which may reduce accuracy in other months.
- Weather-source mismatch: training uses Met Éireann daily min/max aggregates while inference uses OpenWeatherMap hourly point forecasts, introducing a systematic distributional shift in the `avg_*` features.
- The Decision Tree's superior in-sample metrics (MAE 0.970, R^2 0.941) compared to the Random Forest suggest potential for further gains if overfitting can be controlled through deeper regularisation or cross-validation.

Future Improvements

- Expand training history to cover multiple months and seasons for better generalisation.
- Add lag features (e.g., bike count at $t-1$ hour) to exploit temporal autocorrelation.
- Perform systematic hyperparameter optimisation (e.g., grid search or Bayesian optimisation) on the Random Forest.
- Introduce prediction confidence intervals to communicate uncertainty to users.